

SpriteCraft: Texture Style Transfer for Minecraft Resource Packs

A Diffusion-Based Approach to Texture Synthesis and Style Transfer

Ray Zhang

Shanghai Jiao Tong University

June 5, 2026

Abstract

Texture style transfer for pixel art is challenging, due to low resolution amplifying color errors. Diffusion models have shown promise for natural image synthesis but are predominantly trained on photographic datasets and struggle with the discrete palette structures and tileability requirements of game textures. We propose SpriteCraft, a per-pack training framework that ensembles style-conditioned diffusion with a RecolorNet for Minecraft resource pack texture generation. The system achieves good transfer results on a variety of stylized Minecraft resource packs, while being small enough to be rapidly trained on general consumer GPU hardware.

Acknowledgements

The author thanks the Minecraft modding and resource pack community for maintaining open documentation on blockstate models and texture conventions, without which the automated preprocessing pipeline would not have been feasible.

The author also expresses gratitude to the creators and rights holders of the texture sets used in this work. The preprocessing pipeline uses the Minecraft Java Edition 1.21.10 client jar as the vanilla reference pack, and the training / evaluation corpus draws on the following resource packs (versions follow the local archive names in `data/raw_packs`):

- *Ashen 16x*, by Aim_Boot, a 16× medieval-fantasy pack (CurseForge).
- *Bare Bones 1.21.11*, by robotpants, a minimalist pack designed to reproduce the visual style of Minecraft promotional art (CurseForge).
- *Cartoon Texture Pack*, port of the 4J Studios texture pack by BrandonItaly, Lasting Legacy (Modrinth).
- *Chibli 64x Freepack*, by batusaimc, an anime/cartoon-themed 64× pack (CurseForge).
- *Clarity*, by SCTester, a 32× vanilla-adjacent pack (CurseForge).
- *Faithful 32x*, by the Faithful Resource Pack team (faithfulpack.net).
- *Lithos*, credited in-pack to Eleazzaar and friends (CurseForge).
- *MYTHIC*, credited in-pack to FishyMint, a fantasy-themed resource pack (CurseForge).
- *Patric 32x basic*, by Patric (Modrinth).
- *Tooniverse*, by Crafty Fox Digital, a colourful cartoon-style pack (tooniverse.gg).
- *VanillaXBR*, credited in-pack to Misterk7-, an upscaled vanilla-style pack (Modrinth).

This report is typeset using a template adapted from `latex-templates/scientific-thesis-template`, released under the 0BSD license.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Problem Statement	1
1.2 Contributions	2
1.3 Document Structure	2
2 Background	3
2.1 Neural Style Transfer	3
2.2 Diffusion Models	3
2.3 U-Net Architecture	3
2.4 Pixel Art and Low-Resolution Image Synthesis	4
2.5 Optimization	4
3 Methodology	5
3.1 Data Pipeline	5
3.1.1 Full-Cube Texture Filtering	5
3.1.2 Resolution Handling	6
3.1.3 Dataset Construction	6
3.2 Model Architecture	6
3.2.1 StyleAwareUNet	7
3.2.2 RecolorNet	8
3.3 Conditioning and Style Reference Ranking	8
3.4 Training Objective	9
3.4.1 Diffusion Training	9
3.4.2 Content-Aware Losses	10
3.4.3 RecolorNet Training	10
3.4.4 Optimization Details	11
3.5 Inference and Ensemble Rules	11
3.5.1 Diffusion Sampling	12
3.5.2 Candidate Selection	12
3.5.3 Color Transfer Methods	13

4	Experiments	14
4.1	Datasets	14
4.2	Training Configuration	15
4.3	Training Dynamics	16
4.3.1	Overall Loss Curves	16
4.3.2	Loss Breakdown	16
4.3.3	Checkpointed Diffusion Loss	16
4.3.4	Learning Rate Schedule	18
4.4	Evolution of Training Runs	18
4.4.1	Diffusion Baseline	18
4.4.2	Dual-Model Architecture	18
4.4.3	Post-Hoc Quality Evaluation	19
4.4.4	High-Pattern Content Gating	19
4.4.5	Source-Close Loss Routing and Final Configuration	19
4.5	Quantitative Results	20
4.6	Qualitative Analysis	21
4.6.1	What This Model is Suited For	21
4.6.2	What This Model Struggles With	22
4.7	Ablation Studies	22
4.7.1	Selection Policy Versus Oracle Error	22
4.7.2	Detail Injection Usage	23
4.7.3	Pack-Dependent Loss Interaction	23
4.8	Limitations of Current Approach	23
5	Conclusion	25
5.1	Summary of Findings	25
5.2	Limitations	26
5.3	Future Work	26
A	Reproduction Instructions	27
A.1	Environment Setup	27
A.2	Preprocessing	27
A.3	Training	28
A.4	Generation	28
A.5	Repository Layout	29
A.6	Build Notes	29
A.7	Key Configuration Constants	29
	Bibliography	30

List of Figures

3.1	SpriteCraft pipeline overview.	5
3.2	Dual-model architecture.	7
3.3	Ensemble routing flow.	11
4.1	Cross-pack diffusion-only previews from the baseline sweep.	15
4.2	Smoothed total training loss for the four fully trained diffusion packs (log scale). .	16
4.3	Loss breakdown for Ashen 16x training (log scale).	17
4.4	Diffusion training loss sampled every 500 steps from <code>training_metrics.csv</code> for all four packs.	17
4.5	Learning rate schedule.	18
4.6	Selector-decided outputs for <code>oak_planks.png</code> across four target packs.	21
4.7	Ashen_16x routing examples from the current ensemble.	22

List of Tables

- 4.1 Validation texture set. 14
- 4.2 Validated target packs and checkpoint coverage. 14
- 4.3 Training hyperparameters for the final configuration. 15
- 4.4 Diffusion-only baseline sweep, measured from the latest saved validation preview
for each pack. 20
- 4.5 Current ensemble comparison at the final diffusion checkpoint. Cells report MAE /
pixel accuracy. 20
- A.1 Key configuration constants in `src/spritecraft/config.py`. 29

1 Introduction

Texture style transfer aims to preserve the semantic content and spatial structure of an input image while adopting the visual language of a target style domain. In the context of video game graphics, this problem takes on practical significance: resource packs for games like Minecraft consist of hundreds of small, tileable block textures, and manually creating a new pack requires painstaking artistic work for every block. Automating this process while maintaining material identity, edge sharpness, and tiling consistency remains an open challenge.

Recent advances in diffusion-based image generation [1, 4, 10] have produced remarkable results for natural image synthesis. However, these models are predominantly trained on large-scale photographic datasets and assume input resolutions far exceeding the 32×32 pixel grid typical of pixel art textures. When applied directly to block textures, off-the-shelf diffusion pipelines often fail to grasp the style of such low-resolution images. Additionally, such models demand considerable computational resources, making the training process inaccessible for many practitioners.

Our goal is to design a dedicated network that can be trained on a single consumer GPU within a reasonable timeframe, which produces high-quality stylized textures and is able to understand and migrate the style of vanilla Minecraft textures to a variety of target packs.

1.1 Problem Statement

Given a vanilla Minecraft block texture $x_c \in [0, 1]^{3 \times 32 \times 32}$, a target resource pack p , and a small set of style reference textures S_p drawn from pack p , the system must generate a target-style texture y that satisfies three constraints:

- (C1) **Material identity:** the output must retain the block identity of x_c so that a human observer recognizes it as the same material.
- (C2) **Style fidelity:** the color palette, contrast distribution, and textural motifs must belong to the target pack’s visual language.
- (C3) **Tileability:** the output should be visually plausible as a repeating tile in a block-based world, avoiding abrupt boundary discontinuities.

This framing provides paired supervision: since every target pack maps to the same canonical vanilla texture names, each content–target pair constitutes a supervised training sample. This makes the problem a controlled test bed for studying low-resolution image translation, style conditioning from few-shot exemplars, structure preservation, and ensemble-based model selection.

1.2 Contributions

This work makes the following contributions:

- A fully automated data pipeline that extracts, resizes, filters, and pairs full-cube block textures across arbitrary Minecraft resource packs, resolving blockstate models to identify valid full-block textures.
- A dual-model architecture combining a style-conditioned denoising diffusion probabilistic model (StyleAwareUNet, 7.7M parameters) with a structure-preserving recoloring network (RecolorNet, 1.0M parameters), trained per target pack on vanilla-anchored texture pairs.
- A family of content-aware loss functions—structure, contrast, hue, and color moment—with source-relative normalization and target-gated spatial weighting, folded into the diffusion training objective through SNR-weighted x0 prediction.
- A tiered inference pipeline with stochastic diffusion sampling, post-hoc structural quality evaluation, detail injection, and learned recolor fallback, selecting the best candidate based on measured output quality rather than hardcoded routing.
- Qualitative and quantitative evaluation across multiple training runs, documenting the progression from baseline diffusion-only synthesis to the final dual-model system.

1.3 Document Structure

Chapter 2 reviews related work in neural style transfer, diffusion models, and encoder-decoder architectures. Chapter 3 describes the SpriteCraft data pipeline, model architecture, training objective, and inference procedure in detail. Chapter 4 reports training configurations, quantitative metrics, qualitative evaluation, and ablation studies. Chapter 5 summarizes findings, discusses limitations, and outlines directions for future work.

2 Background

2.1 Neural Style Transfer

Pioneered by Gatys et al. [2], neural style transfer separates image representations into content and style components using the feature activations of pre-trained convolutional networks. Subsequent work introduced feed-forward generator networks for real-time stylization [7] and conditional adversarial frameworks for paired image-to-image translation [6]. SpriteCraft occupies an intermediate position: the input-output pairing is supervised (vanilla to target pack), but the style is defined by a small set of reference exemplars rather than a global style representation. Unlike photographic style transfer, the output domain consists of small, tileable game textures where pixel-level structure carries semantic meaning.

2.2 Diffusion Models

Denoising diffusion probabilistic models (DDPMs) [4] learn to reverse a gradual Gaussian noising process, generating samples by iteratively denoising pure noise through a learned Markov chain. Subsequent refinements introduced improved noise schedules [9], classifier-free guidance [1], and latent-space diffusion for high-resolution synthesis [10]. The continuous-time formulation of Song et al. [13] unifies DDPMs with score-based generative models.

For conditional generation, diffusion models can incorporate conditioning signals through cross-attention, concatenation, or feature modulation. Palette [12] demonstrated that a single diffusion framework can unify diverse image-to-image tasks including colorization, inpainting, and uncropping. SpriteCraft adopts epsilon-prediction DDPM with a compact 40-timestep cosine schedule, conditioning on vanilla content, style references, and diffusion timestep through a combination of early fusion, cross-attention, and sinusoidal time embeddings. The 32×32 resolution keeps training tractable but magnifies the importance of detail placement and contrast preservation.

2.3 U-Net Architecture

U-Net [11] combines a contracting encoder path with an expanding decoder path, using skip connections to preserve fine spatial information that would otherwise be lost through bottleneck compression. This architecture has become standard for dense prediction tasks including image segmentation, image-to-image translation, and diffusion-based generation [1]. SpriteCraft employs ResBlock-based [3] encoder-decoder stacks in both models: the StyleAwareUNet for iterative

denoising, and the RecolorNet for deterministic structure-preserving color transfer. The diffusion U-Net additionally incorporates multi-head cross-attention [14] at the bottleneck, allowing content queries to attend across tokenized style reference features.

2.4 Pixel Art and Low-Resolution Image Synthesis

Low-resolution image synthesis presents unique challenges distinct from photographic generation. Pixel art textures typically contain large color contrasts, discrete palette boundaries, and structural motifs (mortar lines, ore flecks, wood grain) whose placement carries semantic meaning. Recent work on pixel art processing includes specialized upscaling approaches [5] and palette-based generation methods. In the Minecraft ecosystem, community-developed tools such as procedural texture generators and color-mapping scripts exist, but they lack the ability to learn per-pack style representations from exemplars. SpriteCraft bridges this gap: by framing the problem as supervised pair-wise style transfer with learned reference selection, the system can adapt to arbitrary pack styles without manual rule definition.

2.5 Optimization

The training loops use AdamW [8], which decouples weight decay from adaptive gradient updates. Both models employ a learning rate schedule combining linear warmup (first 5% of steps, from $0.1\times$ to $1.0\times$ base LR) with cosine annealing to a minimum of 10^{-6} . Exponential moving average (EMA) with decay 0.995 is maintained for both models and used during validation. Gradient clipping at ℓ_2 norm 1.0 and automatic mixed precision (bfloat16) are applied throughout diffusion training.

3 Methodology

3.1 Data Pipeline

The preprocessing pipeline extracts block textures from base (vanilla) and target resource packs, resolves Minecraft blockstate models to identify full-cube block textures, resizes images to a canonical 32×32 resolution, and writes per-pack paired datasets. The pipeline operates on `.zip` or `.jar` archives placed in a raw packs directory.

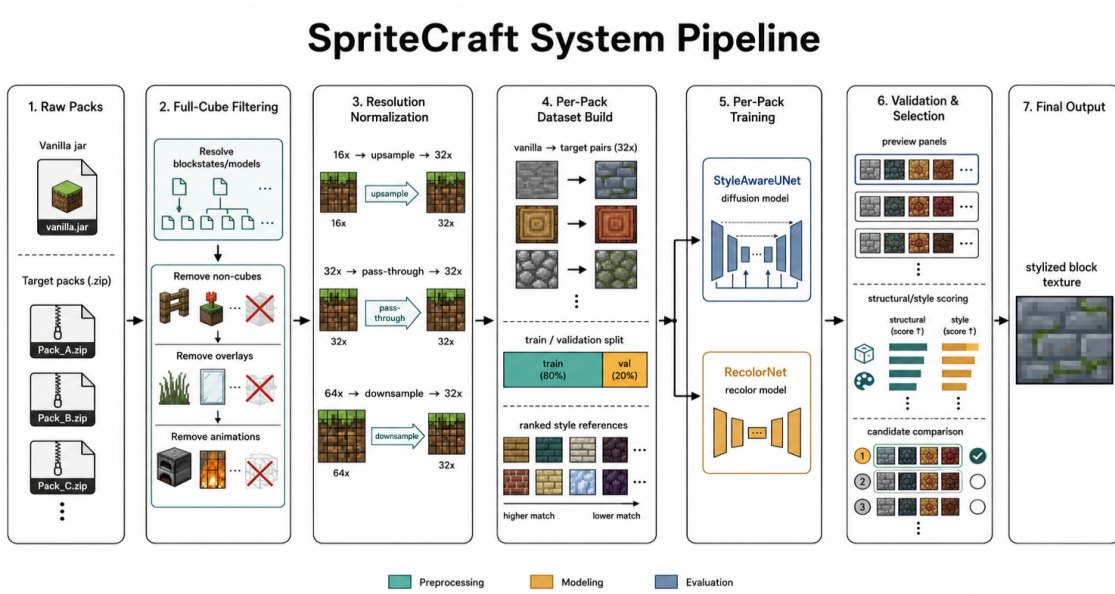


Figure 3.1: SpriteCraft pipeline overview.

The diagram shows raw packs, full-cube filtering, resolution normalization, per-pack dataset construction, per-pack training, validation preview export, and ensemble inference.

3.1.1 Full-Cube Texture Filtering

Not all block textures in a Minecraft resource pack are suitable for training. Many blocks use non-cube models (fences, stairs, slabs, torches), animated textures, or overlay composites. To filter to valid full-cube block textures, the pipeline reads each block's `blockstate.json` file,

resolves model references through parent inheritance chains, and checks whether the resolved model contains a $16 \times 16 \times 16$ cuboid element with all six faces textured. Only textures referenced by such full-cube models are retained. This filtering step is critical: including non-cube textures would introduce structural mismatches between content and target pairs, undermining the learned mapping.

3.1.2 Resolution Handling

Resource packs target three canonical resolutions: $16\times$, $32\times$, and $64\times$ (pixels per block face). The pipeline standardizes all textures to 32×32 :

- $16\times$ images are upscaled using nearest-neighbor interpolation to preserve crisp pixel edges.
- $32\times$ images are used as-is.
- $64\times$ images are downsampled using box (area) filtering rather than Lanczos, to keep texel edges crisper and avoid ringing artifacts.

Images are filtered by aspect ratio (square only), file format (`.png`), and mode (RGBA, RGB, P, or L). Textures with accompanying `.mcmeta` animation files or `_overlay` suffixes are excluded.

3.1.3 Dataset Construction

For each non-base target pack, the pipeline identifies textures whose filenames appear in both the base pack and the target pack (shared textures). These are split into training and validation sets using a fixed validation filename set of 20 canonical block textures (`stone.png`, `oak_planks.png`, `diamond_ore.png`, etc.). The remaining shared textures form the training set. Each target pack also contributes all of its textures (not just shared ones) to a style reference pool for sampling during training and inference.

The final per-pack dataset is stored as a compressed NumPy archive (`dataset.npz`) containing content and target RGB arrays for both splits, plus the full style reference pool. Metadata including filenames, base pack ID, and style labels are recorded in a companion `pair_index.json`.

3.2 Model Architecture

SpriteCraft employs a dual-model architecture: a style-conditioned diffusion model for stochastic generation and a lightweight recoloring network for deterministic structure-preserving color transfer.

The diagram shows the vanilla content texture and ranked style references feeding both StyleAware-UNet and RecolorNet, then the selector / detail-injection stage that chooses the final output.

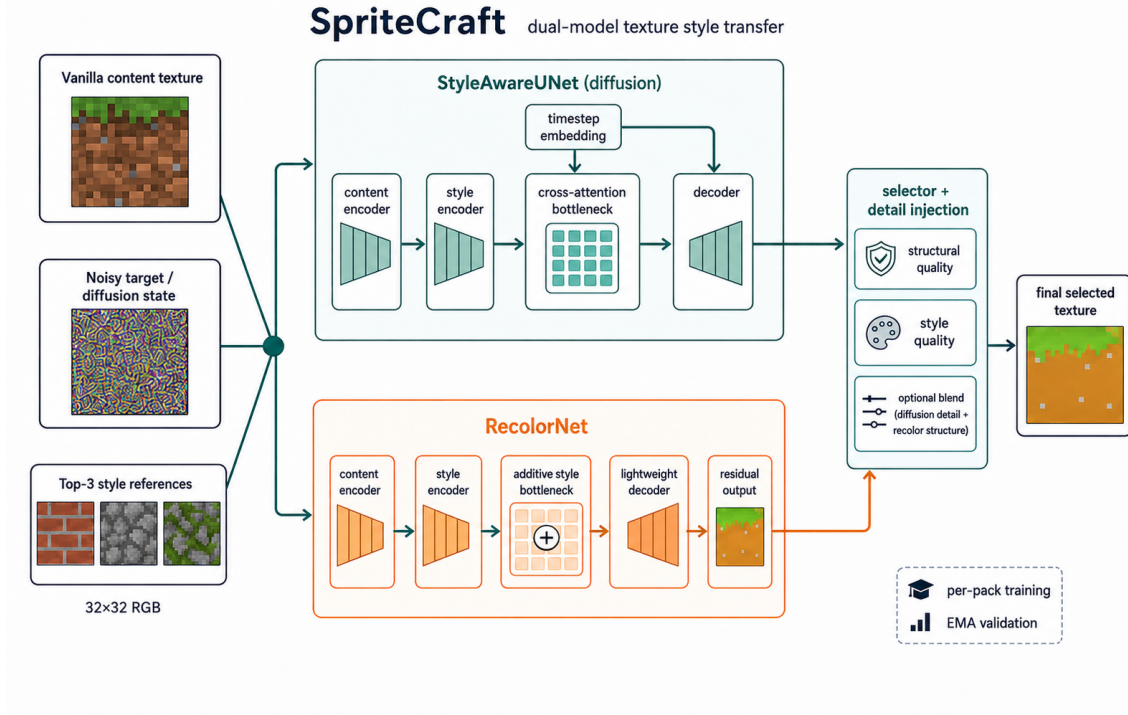


Figure 3.2: Dual-model architecture.

3.2.1 StyleAwareUNet

The primary model is a compact U-Net ($\sim 7.7\text{M}$ parameters) that predicts diffusion noise $\epsilon_{\theta}(x_t, c, s, t)$ given the noisy target x_t , vanilla content c , style references s , and diffusion timestep t . The architecture comprises:

Input fusion. The noisy target and vanilla content share spatial structure (both are $3 \times 32 \times 32$ RGB tensors). They are encoded through separate 3×3 convolutional projections then summed before entering the encoder stack. This early fusion lets the denoiser access both the current noisy state and the vanilla structure at every spatial scale.

Encoder. The encoder compresses the 32×32 representation through two strided convolutional downsampling stages to 8×8 : $32 \rightarrow 16 \rightarrow 8$. Each stage consists of a stride-2 convolution followed by a ResBlock with GroupNorm and SiLU activation. Channel counts double at the first downsampling ($128 \rightarrow 256$) and remain at 256 thereafter.

Style encoding and cross-attention. Each of the $N = 3$ style reference textures passes through a shared style encoder that reduces it to a $64 \times 8 \times 8$ feature map. Rather than mean-pooling across references (which would destroy distinctive per-reference motifs), the style features are concatenated spatially into a single $64 \times 8 \times 8N$ context tensor. At the bottleneck, content features query this style context through multi-head cross-attention ($h = 4$, zero-initialized output projection). A boolean validity mask ensures that padded (absent) references do not contribute to attention weights.

Timestep conditioning. Diffusion timesteps are encoded through sinusoidal embeddings (dimension 64) followed by a two-layer MLP ($64 \rightarrow 256 \rightarrow 256$, SiLU activation), then added to the bottleneck representation.

Decoder. The decoder reconstructs the 32×32 output through two transposed convolution stages ($8 \rightarrow 16 \rightarrow 32$) with ResBlocks. Spatial skip connections from the encoder at 32×32 and 16×16 preserve fine edge structure. The final 1×1 convolution produces a 3-channel noise prediction.

3.2.2 RecolorNet

The RecolorNet ($\sim 1.0\text{M}$ parameters) is a lightweight U-Net designed for structure-preserving color transfer. It takes the vanilla content texture and style references as input and predicts a residual added to the content, producing a recolored output in $[0, 1]$. The architecture follows a simpler U-Net pattern with fewer channels (base 64) and no diffusion components:

- The content encoder compresses the vanilla input to $64 \times 8 \times 8$ through two downsampling stages.
- The style encoder (shared architecture with the diffusion U-Net’s style encoder) processes reference textures, whose features are averaged with a validity mask and projected to the content feature space.
- Style conditioning is injected additively at the bottleneck.
- A decoder with transposed convolutions upscales to 32×32 , with a direct skip connection to the raw content RGB at the final merge stage.
- The output convolution is zero-initialized, making the initial prediction a pure copy of the content.

3.3 Conditioning and Style Reference Ranking

Style reference selection is critical: given a target texture to generate, the model should be conditioned on exemplars from the same pack that are visually relevant to the task. SpriteCraft employs a heuristic ranking system based on two components:

Content descriptors. Each texture is represented by a normalized descriptor vector combining gradient orientation histograms (8 bins), per-channel color histograms (4 bins $\times$ 3 channels), radial FFT energy features (4 bands), and statistical moments (mean, standard deviation, gradient energy, entropy). Descriptors are L2-normalized for cosine-similarity comparison.

Filename priors. Texture filenames carry semantic information. Families (wood, foliage, ore, brick, glass, soil, stone, ceramic) and surface roles (top, bottom, side, end, core) are inferred from lexical token matching against predefined rules. Shared material tokens between candidate and

target receive a similarity bonus; extraneous color tokens (e.g., “red” in “red_sand” when the target is “sand”) receive a penalty.

The final ranking sorts candidates by composite score: descriptor cosine similarity plus token-based bonuses minus mis-match penalties. Training and inference both use the top- N ranked candidates as style references, with $N = 3$ in the current configuration.

3.4 Training Objective

3.4.1 Diffusion Training

The StyleAwareUNet is trained to predict the noise ϵ that was added to a clean target x_0 at timestep t :

$$\mathcal{L}_{\text{noise}} = \mathbb{E}_{x_0, c, s, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, c, s, t)\|_2^2] \quad (3.1)$$

where $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ and $\bar{\alpha}_t$ follows a cosine schedule ($s = 0.008$, $\beta_{\max} = 0.999$) with $T = 40$ timesteps. The cosine schedule ensures near-zero terminal signal-to-noise ratio, preventing the model from exploiting residual clean signal at the terminal timestep.

From the predicted noise, the clean target estimate $\hat{x}_0$ is recovered via:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta}{\sqrt{\bar{\alpha}_t}} \quad (3.2)$$

Auxiliary losses are computed on $\hat{x}_0$ and weighted by the signal-to-noise ratio $\bar{\alpha}_t$ so that at high noise levels (where $\hat{x}_0$ is unreliable), the model focuses on noise prediction rather than reconstruction:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{noise}} + \bar{\alpha}_t \cdot (0.70 \mathcal{L}_{\text{recon}} + 0.30 \mathcal{L}_{\text{gradient}} + 0.20 \mathcal{L}_{\text{content}}) \quad (3.3)$$

where $\mathcal{L}_{\text{recon}}$ is L1 reconstruction loss on $\hat{x}_0$ and $\mathcal{L}_{\text{gradient}}$ combines luminance forward-difference L1 (preserving edge sharpness) with per-channel RGB gradient L1 at weight 0.5 (penalizing color bleeding).

3.4.2 Content-Aware Losses

Content-aware losses support the diffusion objective by directly supervising structural and color fidelity. All structural comparisons are *source-relative*: they measure deltas from the vanilla content rather than absolute target values, so the model learns to predict only what changes:

$$\mathcal{L}_{\text{content}} = 1.0 \mathcal{L}_{\text{structure}} + 2.5 \mathcal{L}_{\text{contrast}} + 3.0 \mathcal{L}_{\text{hue}} + 1.5 \mathcal{L}_{\text{color_moment}} \quad (3.4)$$

Structure loss combines three source-relative delta terms:

- Gradient delta: L1 between prediction’s gradient change and target’s gradient change.
- Detail delta (weight 0.5): L1 on high-pass residuals relative to content.
- Laplacian delta (weight 0.75): L1 on 4-neighbor Laplacian response deltas.

Contrast loss applies a target-gated, one-sided under-contrast penalty. Local luminance standard deviation σ is estimated via windowed pooling (kernel sizes 3 and 5, averaged). A spatial gate $g = (\sigma_{\text{target}} / \max(\sigma_{\text{target}}))^{\gamma}$ with $\gamma = 1.5$ focuses the penalty on high-detail regions. Critically, only under-contrast is penalized ($\text{ReLU}(\sigma_{\text{target}} - \sigma_{\text{pred}})$); over-sharpening is already constrained by reconstruction and structure losses. An edge shortfall term ($\beta = 0.6$) additionally penalizes gradient magnitudes that fall below the target’s.

Hue loss operates in opponent-chroma space to avoid HSV discontinuities. Chroma vectors $(R - G, B - 0.5(R + G))$ are L2-normalized to unit direction. Angular misalignment is gated by the target’s chroma magnitude ($\gamma = 0.7$), so flat gray regions are ignored. A chroma strength term (weight 0.25) aligns saturation levels.

Color moment loss matches per-channel mean and standard deviation between prediction and target (weight 0.5 on std). This non-spatial term provides a strong, reliable global color gradient that converges quickly.

A detail emphasis factor ($\gamma = 2.0$) upweights all spatial losses for textures whose targets contain concentrated detail (top 12.5% of local std values). A gate floor of 0.05 ensures all regions contribute at least minimal gradient.

3.4.3 RecolorNet Training

RecolorNet is trained with a simpler objective combining four terms:

$$\mathcal{L}_{\text{recolor}} = \mathcal{L}_{\text{recon}} + 0.5 \mathcal{L}_{\text{edge}} + 1.5 \mathcal{L}_{\text{color}} + 0.1 \mathcal{L}_{\text{identity}} \quad (3.5)$$

Reconstruction L1 drives the prediction toward the target. Edge L1 matches the luminance edge map between prediction and content, enforcing structure preservation. Color moment L1 matches per-channel mean and standard deviation to the target. Content identity L1 anchors the prediction

to the content, preventing the model from drifting into implausible outputs when style evidence is weak.

3.4.4 Optimization Details

Both models use AdamW with learning rate 3×10^{-4} and weight decay 0.01 (RecolorNet uses 0.03). The learning rate follows a linear warmup of 5% of total steps (start factor 0.1), then cosine annealing to 10^{-6} . The diffusion model uses gradient accumulation ($4\times$ on CUDA, batch size 4 effective 16), gradient clipping ($\ell_2 = 1.0$), and bfloat16 mixed precision. Exponential moving average (EMA, decay 0.995) is maintained for both models. Validation uses EMA weights. Training is per target pack, with default steps of 10,000 for the diffusion model and 2,500 for RecolorNet.

3.5 Inference and Ensemble Rules

Inference follows a tiered procedure that generates multiple candidates and selects the best based on measured output quality rather than hardcoded routing:

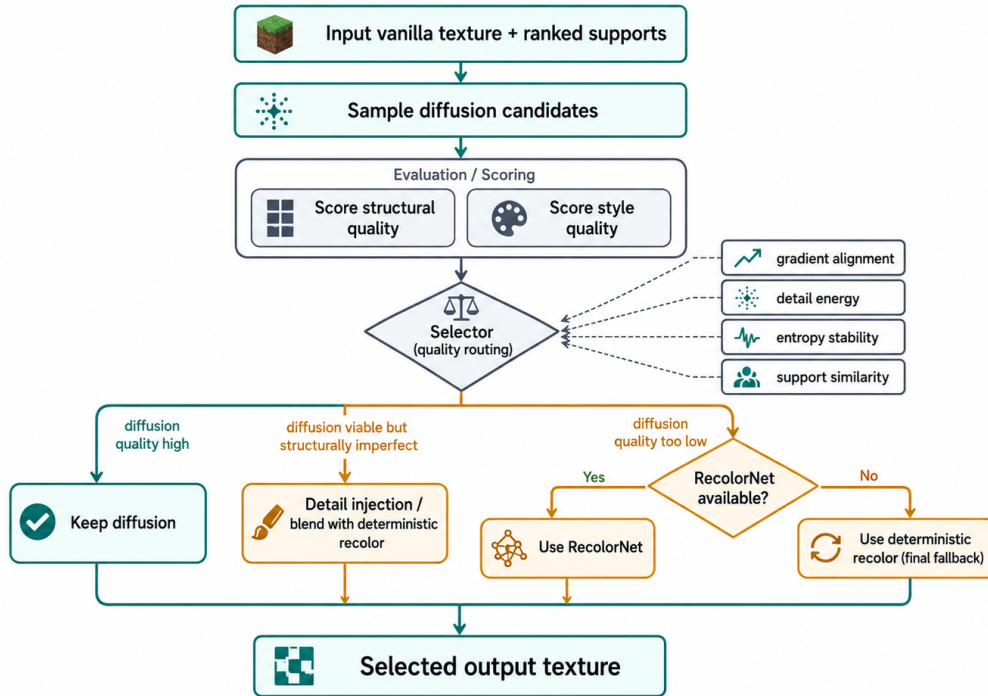


Figure 3.3: Ensemble routing flow.

The flowchart covers diffusion sampling, structural-style scoring, detail injection, recolor fallback, and final candidate selection.

3.5.1 Diffusion Sampling

The reverse process starts from pure Gaussian noise and applies 40 DDPM steps with x_0 clipping to $[0, 1]$:

$$x_{t-1} = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\hat{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \sigma_t z \quad (3.6)$$

where $\hat{x}_0 = \text{clamp}(\frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_\theta}{\sqrt{\bar{\alpha}_t}}, 0, 1)$ and $\sigma_t^2 = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$ for $t > 1$, zero otherwise.

3.5.2 Candidate Selection

Generation produces $N = 2$ stochastic diffusion candidates plus optional RecolorNet and deterministic recolor candidates. Each candidate is scored along two axes:

- **Structural quality:** a post-hoc evaluator computes gradient alignment, edge fidelity (penalizing both loss and explosion), detail coherence (cosine similarity of high-pass residuals), detail energy fidelity (symmetric: penalizing blur below $0.3\times$ and noise above $1.3\times$ content energy), and entropy stability. The weighted composite score (0–1, with 0.03 baseline) is compared against a content-complexity-adaptive threshold.
- **Style quality:** combined from a descriptor-based support similarity score and a source-relative change agreement score. The support descriptor score compares the candidate’s descriptor via cosine similarity to the mean descriptor of active support references, with a sharpness penalty. The change agreement score evaluates how closely the candidate’s *change from vanilla* matches the support pack’s typical change magnitude, penalizing recolor outputs that stay too close to vanilla on stylized packs. The two scores are blended 0.65 / 0.35.

High-pattern content gate. Textures with strong line art, glyphs, or repeated narrow bands (e.g. `tnt.side.png`, `bookshelf.png`) require stricter structural screening. When the content texture exceeds thresholds on edge energy (≥ 0.09), detail energy (≥ 0.05), and complexity (≥ 0.50), the evaluator applies a multiplicative gate to the quality score that enforces tighter edge/detail energy ratios (windows $[0.78, 1.18]$ and $[0.80, 1.16]$) and requires positive detail coherence. The acceptance threshold is also raised to 0.68 for these textures.

Detail injection (non-destructive). When a deterministic recolor is available, it is blended with the diffusion prediction at two amounts (0.35 and 0.65). Unlike earlier approaches that rebuilt the output from a blurred diffusion base plus source detail (causing global softening), the current injection starts from the original diffusion prediction and only adds missing high-frequency residuals from the recolor where: (1) the source has stronger local detail than the diffusion, (2) the low-frequency layouts agree, and (3) the detail vectors are aligned. This local-gated fusion preserves diffusion detail that is already good.

The selection logic proceeds in three tiers:

Tier 1: Diffusion viability. If the best diffusion candidate has structural quality ≥ 0.80 , it is confidently preferred. If quality is ≥ 0.60 (viable), a clear margin of 0.10 in style score must favor the alternative to override.

Tier 2: Detail-injected candidates. Blended candidates replace the raw diffusion prediction only if they surpass it on the combined structural + style score with a 0.01 margin *and* preserve at least 97% of the diffusion’s edge and high-pass energy (preventing over-softening from the injection).

Tier 3: Recolor fallback. When structural quality is below 0.60, the diffusion output is rejected and the RecolorNet output is used. The deterministic recolor serves as a final fallback when RecolorNet is unavailable.

3.5.3 Color Transfer Methods

Two deterministic color transfer methods serve as baselines and fallback candidates:

Support-pair affine recolor. When vanilla-side references are available for the style references (the same texture names appear in both packs), per-channel affine transforms are fitted: $\text{target}_{\text{ch}} = \text{scale}_{\text{ch}} \times \text{source}_{\text{ch}} + \text{bias}_{\text{ch}}$. The scale is clamped to $[0.2, 3.0]$. The fitted transform is applied to the vanilla content.

Style-stat recolor. When support pairs are unavailable, a simpler transformation matches per-channel mean and standard deviation of the style references to the content.

These deterministic methods serve as the final safety net when both diffusion and RecolorNet produce unsatisfactory outputs.

4 Experiments

4.1 Datasets

Experiments were conducted using the vanilla Minecraft 1.21 jar as the base (content) pack. The evaluation corpus spans multiple target packs rather than a single Ashen-only case study. This report compares a historical diffusion-only sweep with the current dual-model ensemble, which combines StyleAwareUNet, RecolorNet, deterministic recoloring, support-aware candidate selection, and source-close loss routing. The target packs cover minimalist, vanilla-adjacent, cartoon, fantasy, and higher-resolution stylized packs, so the report can describe both general trends and pack-specific failure modes.

Table 4.1: Validation texture set.

stone	dirt	cobblestone	oak_planks
spruce_planks	sand	gravel	gold_ore
iron_ore	coal_ore	diamond_ore	oak_log
oak_leaves	glass	farmland	bricks
tnt_side	bookshelf	mossy_cobblestone	obsidian

Table 4.2: Validated target packs and checkpoint coverage.

Pack	Style	Train	Val	Baseline sweep	Current ensemble
Ashen_16x	stylized	509	19	diffusion to 10.0k	diffusion 10.0k + recolor 2.5k
Bare Bones	minimalist	499	18	diffusion to 10.0k	diffusion 10.0k
Cartoon	cartoon	404	19	diffusion to 5.5k	diffusion 10.0k + recolor 2.5k
Chibli 64x	stylized	87	17	diffusion to 0.5k	diffusion 10.0k + recolor 2.5k
Clarity	traditional	493	19	diffusion to 0.5k	recolor only to 2.5k
Faithful 32x	vanilla	519	19	diffusion to 0.5k	–
Lithos	traditional	497	19	diffusion to 0.5k	–

The MYTHIC training branch was started during the broad diffusion-only sweep, but no validation preview panels were written before the run stopped, so it is excluded from the quantitative comparison. Figure 4.1 shows the breadth of that sweep at a single shared validation texture: even at an early preview step, the model is already copying broad stone-material contrast into several packs, but it remains noticeably closer to vanilla on the more stylized and high-detail targets.

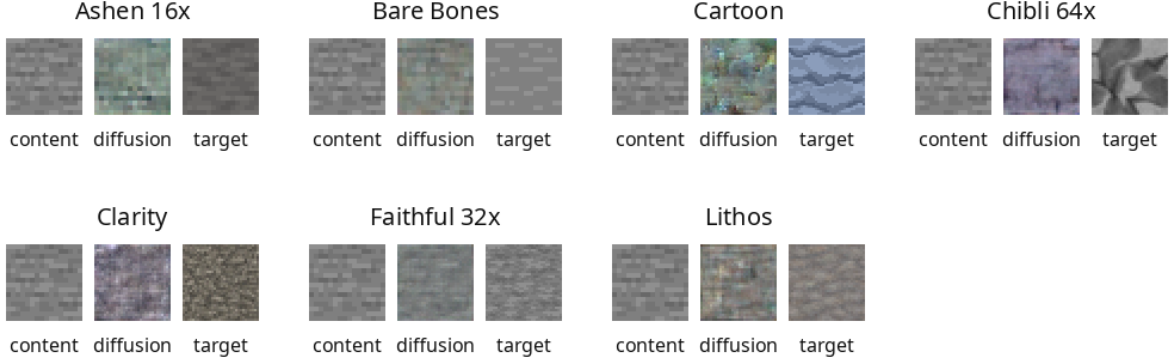


Figure 4.1: Cross-pack diffusion-only previews from the baseline sweep.

4.2 Training Configuration

Both models were trained on a single NVIDIA GPU with bfloat16 autocast. The diffusion model (StyleAwareUNet, 7.7M parameters) was trained for 10,000 steps with gradient accumulation of 4 (effective batch size 16), while RecolorNet (1.0M parameters) was trained for 2,500 steps where a recolor branch was available. Full configuration details are summarized in Table 4.3.

Table 4.3: Training hyperparameters for the final configuration.

Parameter	Diffusion (StyleAwareUNet)	RecolorNet
Optimizer	AdamW	AdamW
Learning rate	3×10^{-4}	3×10^{-4}
Weight decay	0.01	0.03
LR schedule	Warmup (5%) + Cosine	Warmup (5%) + Cosine
Min LR	10^{-6}	10^{-6}
Batch size (per step)	4	8
Gradient accumulation	4	—
Effective batch size	16	8
Gradient clip (ℓ_2)	1.0	—
Mixed precision	bfloat16	—
EMA decay	0.995	0.995
Training steps	10,000	2,500
Diffusion timesteps	40	—
Beta schedule	Cosine ($s = 0.008$)	—
Style references (N)	3	3

4.3 Training Dynamics

4.3.1 Overall Loss Curves

Figure 4.2 shows the smoothed training loss trajectories for the four packs with fully trained diffusion branches. Bare Bones converges fastest and to the lowest final loss, consistent with its minimalist style and the fact that more than half of its minibatch samples route through the source-close legacy objective. Ashen and Cartoon follow similar trajectories with a wider initial spread. Chibli, with its smallest training set (87 pairs), converges more slowly and shows higher variance throughout training.

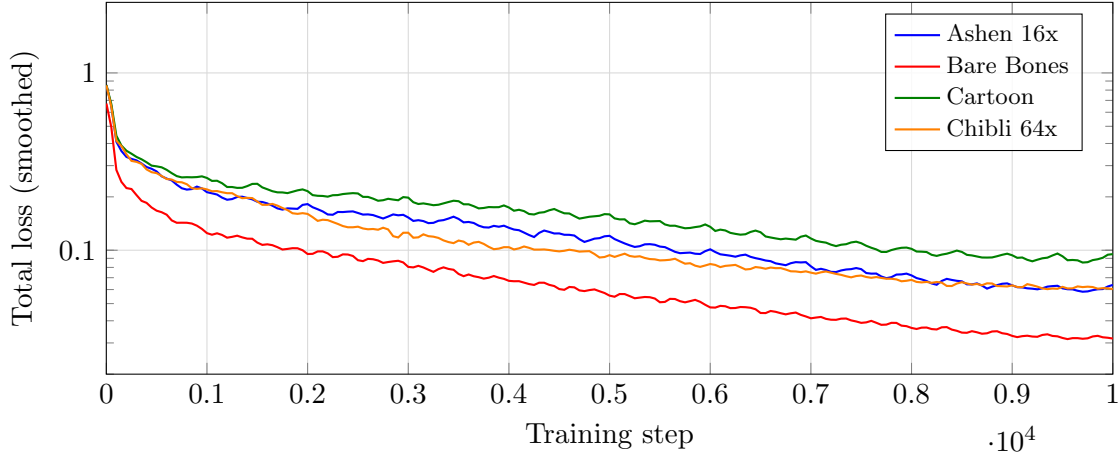


Figure 4.2: Smoothed total training loss for the four fully trained diffusion packs (log scale).

4.3.2 Loss Breakdown

Figure 4.3 decomposes the Ashen 16x training loss into its constituent terms. The noise prediction loss dominates early training but declines rapidly. Content-aware losses (structure, contrast, hue, color moment) decay more gradually and account for much of the remaining loss at convergence. The gradient loss persists modestly, reflecting the inherent difficulty of edge placement in stylized packs.

4.3.3 Checkpointed Diffusion Loss

Figure 4.4 plots the diffusion model’s raw training loss sampled every 500 steps, matching the checkpoint cadence used in the final training configuration. This avoids a reporting artifact in the routed validation summaries: on hard packs, selected-output validation can flatten once recolor fallback dominates, even when the diffusion branch is still struggling. Bare Bones drops fastest and remains lowest throughout. Chibli stays highest for most of training, while Ashen and Cartoon show a slower but still steady decline with occasional spikes that reflect the harder stylized targets.

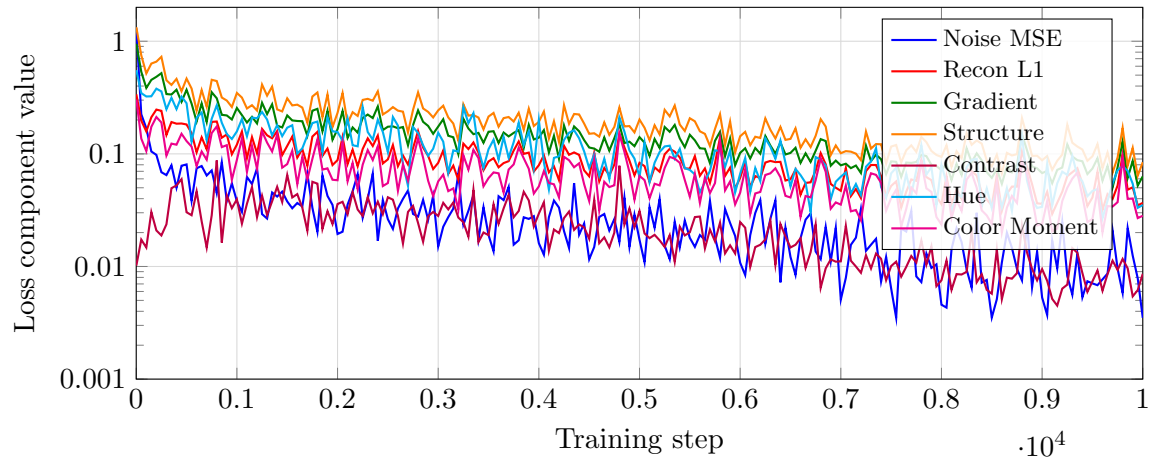


Figure 4.3: Loss breakdown for Ashen 16x training (log scale).

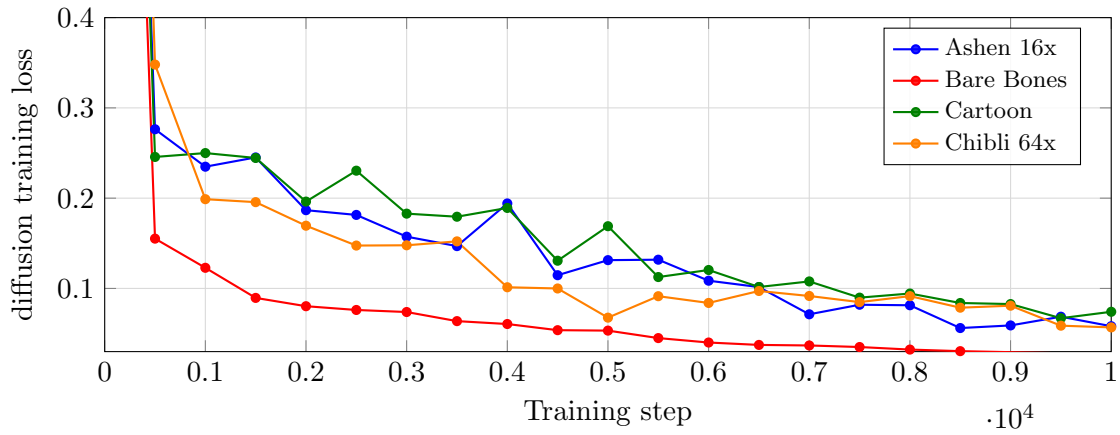


Figure 4.4: Diffusion training loss sampled every 500 steps from `training_metrics.csv` for all four packs.

4.3.4 Learning Rate Schedule

Figure 4.5 confirms the warmup + cosine annealing schedule. The first 500 steps warm up linearly from 3×10^{-5} to the peak of 3×10^{-4} , after which cosine annealing smoothly decays the rate to the floor of 10^{-6} at step 10,000.

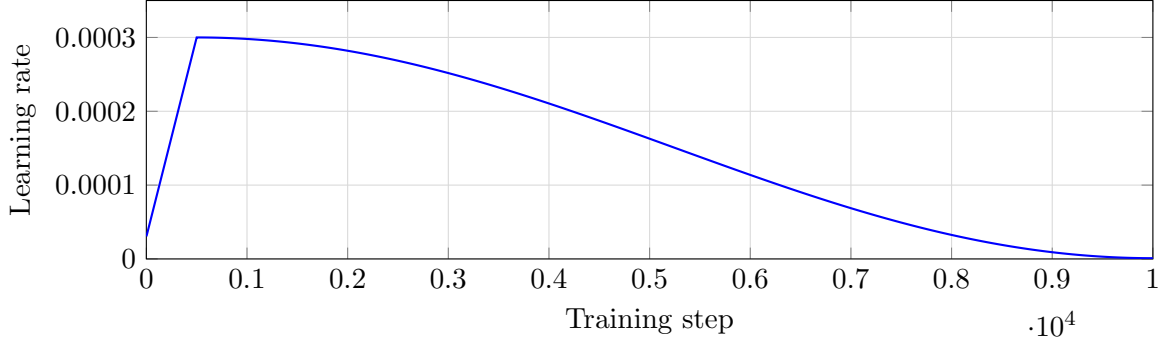


Figure 4.5: Learning rate schedule.

4.4 Evolution of Training Runs

4.4.1 Diffusion Baseline

The initial wide sweep used a diffusion-only model with a linear noise schedule and base channel count 128. No post-hoc selector or recolor fallback was available. The main value of this baseline is not a single headline number, but the fact that the same model family was exposed to seven distinct pack styles. Minimalist and vanilla-adjacent targets converged quickly: Bare Bones and Faithful already looked coherent at their latest preview steps, while stylized packs with stronger palette shifts such as Ashen, Cartoon, and Chibli showed more visible drift, especially on noisy materials and fine outlines. This broad run established that *pack difficulty is not uniform*; the training recipe that is comfortable for Bare Bones is not automatically adequate for a $64 \times$ stylized pack compressed to 32×32 .

4.4.2 Dual-Model Architecture

The next iteration introduced RecolorNet as a structural fallback together with the content-aware loss suite (structure, contrast, hue, color moment). This reduced hue instability and helped the diffusion branch preserve stronger edge structure, but it also made clear that a fixed routing threshold was too crude across multiple packs. Hard textures such as `bookshelf.png` and `tnt_side.png` benefit from deterministic structure preservation, whereas easy textures in minimalist packs can be handled by diffusion alone. A pack-blind selector therefore risks being either too permissive on difficult packs or too conservative on easy ones.

4.4.3 Post-Hoc Quality Evaluation

The next iteration replaced the fixed routing logic with the structural-style evaluator described in Section 3.5. This evaluator scores the actual diffusion output against the content image and its support set, then decides whether to keep diffusion, apply detail injection, or fall back to recolor. Training improvements included EMA (decay 0.995), linear learning-rate warmup over 5% of steps, and an adaptive content-loss ramp over the early phase of training.

4.4.4 High-Pattern Content Gating

The post-hoc evaluator was further refined to detect and tightly screen *high-pattern content*—textures with strong line art, symbolic glyphs, or repeated narrow bands such as `tnt_side.png` and `bookshelf.png`. When the content texture exceeds thresholds on edge energy, detail energy, and complexity, the evaluator applies a multiplicative gate ($0.55 + 0.45 \times \text{gate_score}$) to the quality score and raises the acceptance threshold to 0.68. The gate score combines tighter edge ratio fidelity (window $[0.78, 1.18]$), detail energy ratio fidelity (window $[0.80, 1.16]$), and a positive detail coherence gate. This prevents the selector from accepting diffusion outputs that pass the general quality threshold but are structurally unsuitable for precision-dependent textures. The source-relative change agreement score was also added at this stage, blending the support descriptor comparison with a term that rewards candidates whose change-from-vanilla magnitude matches the support pack’s typical change level.

4.4.5 Source-Close Loss Routing and Final Configuration

The final configuration introduced SNR-weighted x_0 auxiliary losses, meaning reconstruction and content losses are scaled by $\bar{\alpha}_t$, the signal-to-noise ratio at the current diffusion timestep. This prevents the model from over-committing to unreliable x_0 estimates at high noise levels. The content loss weight was reduced to 0.20 and the content-loss components were rebalanced to weights 1.0 / 2.5 / 3.0 / 1.5 for structure / contrast / hue / color-moment.

The most important later correction was to route source-close targets back to a legacy direct objective. A training sample is considered source-close when the target/content RGB MAE is at most 0.045. Those samples use noise prediction plus reconstruction and luminance-gradient terms only, dropping the content, color, and per-channel gradient auxiliaries. This isolates packs and textures that are already close to vanilla. In the current training logs, Bare Bones routes approximately 53% of minibatch samples through this path, while Ashen, Cartoon, and Chibli route only about 2–5%. The heuristic therefore captures the easy-pack case without hard-coding a specific pack name.

Additional refinements included:

- Symmetric detail energy fidelity in the post-hoc evaluator.
- Combined candidate selection using structural and style quality scores.

- Wood-family routing that ranks diffusion candidates primarily by support-style agreement.
- Disabled detail injection for wood textures, where the injection path had introduced plank-line artifacts.
- Conservative RecolorNet fallback for wood, used only when diffusion is structurally broken or clearly worse.

4.5 Quantitative Results

The checkpoint folders do not store one uniform metric export for every validated pack, but they do store preview panels with the prediction and target tiles side by side. For the pack-level comparison below, MAE and pixel accuracy were recomputed directly from those panels by cropping the prediction and ground-truth target tiles. For selected-output panels, the colored selection border was removed before computing metrics. Pixel accuracy is reported as the proportion of RGB channel values within 5% of target intensity.

Table 4.4: Diffusion-only baseline sweep, measured from the latest saved validation preview for each pack.

Pack	Latest preview	MAE	Pixel accuracy
Ashen_16x	10.0k	0.138	0.222
Bare Bones	10.0k	0.053	0.665
Cartoon	5.5k	0.154	0.215
Chibli 64x	0.5k	0.172	0.201
Clarity	0.5k	0.150	0.214
Faithful 32x	0.5k	0.091	0.488
Lithos	0.5k	0.111	0.358

Table 4.5: Current ensemble comparison at the final diffusion checkpoint. Cells report MAE / pixel accuracy.

Pack	Diffusion	Recolor	Selected
Ashen_16x	0.110 / 0.321	0.108 / 0.385	0.108 / 0.389
Bare Bones	0.055 / 0.601	–	0.057 / 0.561
Cartoon	0.161 / 0.234	0.173 / 0.234	0.141 / 0.273
Chibli 64x	0.125 / 0.338	0.170 / 0.218	0.164 / 0.242

Several conclusions follow immediately from Tables 4.4 and 4.5. Bare Bones is by far the easiest target family: it already performs best in the diffusion-only baseline, and the current source-close route preserves that strength without requiring pack-specific loss constants. Ashen benefits meaningfully from the later configuration, improving from 0.138 / 0.222 in the baseline to 0.108 / 0.389 in the selected current output. Cartoon improves through ensemble selection even though its raw diffusion branch is not numerically better than the baseline. Chibli remains the hardest pack

because it combines the smallest dataset (87 train pairs) with the most aggressive $64\times \rightarrow 32\times 32$ downscaling. Clarity’s recolor-only branch reaches 0.125 MAE and 0.330 pixel accuracy at the latest saved recolor preview, which is promising but not directly comparable to the four fully trained diffusion runs.

The selector is also more conservative than a pure target-aware MAE oracle. At the final checkpoint, Ashen uses recolor for 15 of 19 selected previews and blend candidates for the remaining 4. Cartoon uses blends for 15 of 19 textures, which explains why its selected score improves over both individual branches. Chibli still falls back mostly to recolor, while Bare Bones has no saved learned-recolor panels at the final checkpoint and all selected previews are diffusion-derived blends.

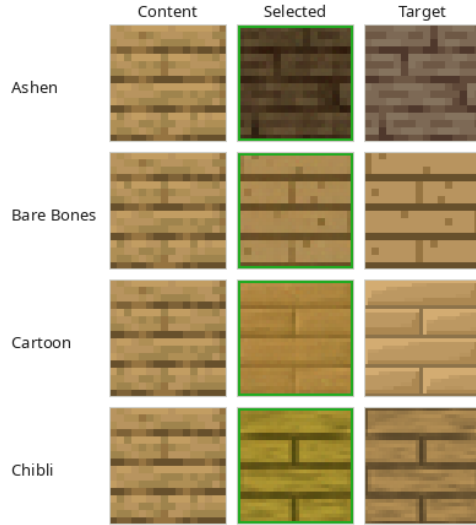


Figure 4.6: Selector-decided outputs for `oak_planks.png` across four target packs.

4.6 Qualitative Analysis

4.6.1 What This Model is Suited For

Vanilla-adjacent and minimalist packs. Bare Bones, Faithful, and parts of Lithos are structurally close enough to vanilla that the system can focus on palette and contrast remapping rather than inventing new geometry. This is why Bare Bones dominates the diffusion-only baseline and remains clean under the current source-close routing.

Medium-complexity stylized materials. Ashen and Cartoon benefit from the content-aware loss suite because they require both strong palette movement and controlled edge placement. Wood, stone, and ore textures usually preserve the correct large-scale structure while moving into the correct pack-specific value range.

Wood-family textures. Wood and plank textures are a special case: their quality depends on long, regular grain and plank-line continuity. The current selector treats them as source-close

style-transfer cases, skips detail injection, and allows the best diffusion candidate to win unless it is clearly broken. This restores the sharper diffusion-like plank structure while keeping the recolor fallback available for genuinely failed samples.

4.6.2 What This Model Struggles With

High-resolution packs compressed to 32×32 . Chibli is the clearest example. The pack has only 87 training pairs after preprocessing and starts from $64 \times$ artwork. Even when the diffusion candidate has respectable overall style, the downscaled microtexture and line work remain difficult to preserve.

Transparent or pseudo-transparent materials. Glass remains a fundamental problem because alpha is discarded before modeling. The network must hallucinate a fully opaque RGB realization of a texture whose appearance in the target pack is often driven more by transparency treatment than by raw color.

Fine symbolic detail. `tnt_side.png` and `bookshelf.png` remain good stress tests because they require preserving legible letters or narrow repeated book spines. In those cases, the selector usually prefers recolor or blend candidates over raw diffusion.

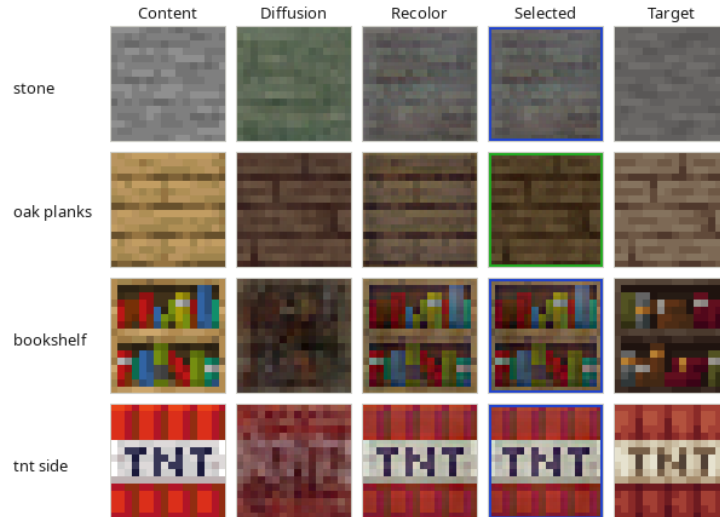


Figure 4.7: Ashen_16x routing examples from the current ensemble.

4.7 Ablation Studies

4.7.1 Selection Policy Versus Oracle Error

The cross-pack comparison reveals a useful tension: the selector is not optimizing target-aware MAE directly. Bare Bones and Chibli both show lower average MAE for the raw diffusion branch

than for the selected output, but the selected outputs are structurally safer and more regular. This is not necessarily a bug; it reflects the design goal of preferring plausible, tileable textures over target-aware oracle error. It does mean, however, that future work should calibrate the selector separately for easy packs and hard packs rather than using one conservative policy everywhere.

4.7.2 Detail Injection Usage

The saved selected panels show that detail injection and blend selection are not rare corner cases. In the current checkpoint, all 18 Bare Bones selections and 15 of 19 Cartoon selections differ from the raw diffusion preview, indicating diffusion-derived blend candidates. On Ashen and Chibli, by contrast, the selector often settles on the pure recolor output. This suggests the blend path is most valuable when the pack remains close enough to vanilla for diffusion detail to be trusted locally.

The non-destructive detail injection (Section 3.5) was critical for this result on non-wood textures. Earlier variants that rebuilt the output from a blurred diffusion base caused global softening on already good diffusion samples. The current approach adds only missing high-frequency residuals where local layout agreement and detail alignment gates permit, preserving diffusion detail that is already plausible.

4.7.3 Pack-Dependent Loss Interaction

The multi-pack comparison supports the concern raised during development that extra structural penalties can backfire on very easy packs. Bare Bones is already strong in the diffusion-only baseline; a flat content-loss recipe therefore makes it optimize auxiliary targets that are less useful than direct reconstruction. The source-close target/content MAE heuristic addresses this without hard-coding Bare Bones: packs and textures close to vanilla use the direct objective, while stylized packs still receive the full content-aware loss. Ashen is the opposite case, and benefits clearly from the added structure and hue guidance.

4.8 Limitations of Current Approach

No alpha-channel modeling. Despite preprocessing and storing alpha channels in the dataset, the model operates on RGB only. Transparent textures (glass, ice, nether portal) are inherently ill-posed in RGB space.

Heuristic support ranking. The descriptor-based ranking with filename priors is effective but brittle. Packs using nonstandard naming conventions or novel material families may receive suboptimal support assignments. A learned ranking model could improve generalization.

Pack-specific training. Each target pack requires a separate training run. While this enables pack-specific specialization (which is the project’s stated goal), it precludes zero-shot generalization to unseen packs.

Resolution ceiling. The fixed 32×32 resolution limits detail fidelity for packs with complex microtexture (leaves, bookshelves). A multi-resolution training scheme could address this but would increase computational cost proportionally.

Evaluation artifacts are preview-first. Several runs store validation as composite preview panels rather than as standalone metric JSON files for every pack and every candidate. That is adequate for human inspection and for the post-hoc analysis performed here, but a future report-oriented pipeline should export raw per-pack metrics directly to avoid reconstruction work.

5 Conclusion

This work presented SpriteCraft, a dual-model system for texture style transfer between vanilla Minecraft block textures and target resource-pack variants. The system combines a style-conditioned denoising diffusion probabilistic model (StyleAwareUNet) with a lightweight structure-preserving recoloring network (RecolorNet), trained per target pack on vanilla-anchored texture pairs.

5.1 Summary of Findings

The central empirical finding is that combining stochastic diffusion-based generation with deterministic structure-preserving recoloring yields a robust pipeline where neither component alone suffices. Diffusion models excel at producing visually rich, style-consistent outputs for structurally simple textures, but they struggle with precise pixel-level placement for complex patterns (TNT lettering, leaf clusters). The RecolorNet provides structurally perfect results at the cost of color expressiveness. By evaluating the *actual output quality* of both models through a post-hoc structural evaluator and selecting based on measured traits, the tiered ensemble achieves the best of both worlds: diffusion is used when its outputs are structurally sound, and RecolorNet takes over when diffusion fails.

Key design choices that proved effective:

- **Source-relative loss:** Computing structural deltas from the vanilla content rather than absolute target values focuses the model on learning what changes, accelerating convergence and improving fidelity.
- **Target-gated losses:** Spatially gating contrast and hue penalties by the target’s local energy prevents the model from hallucinating detail in flat regions while concentrating supervision on high-detail areas.
- **SNR-weighted x0 losses:** Weighting auxiliary reconstruction losses by the signal-to-noise ratio at each diffusion timestep eliminates a hyperparameter trade-off between content loss weight and convergence speed.
- **Post-hoc quality evaluation:** Replacing hardcoded routing rules with output-driven model selection correctly identifies failure cases in real time, routing complex textures to the appropriate model without per-texture hand-tuning.

- **Multi-reference attention with token preservation:** Encoding each style reference as a separate token rather than mean-pooling preserves distinctive per-reference motifs that averaging would destroy.

5.2 Limitations

The current system has several known limitations. Alpha-channel information is preprocessed and stored but not used by the model, making transparent textures (glass, ice) inherently difficult. Style reference ranking relies on heuristic descriptors and filename parsing rather than learned embeddings, limiting adaptability to packs with nonstandard naming conventions. Training is per-pack, so each new target pack requires a full training cycle—acceptable for the project’s stated goal of providing a *framework for rapid per-pack training* but not suitable for zero-shot generalization. The fixed 32×32 resolution ceiling limits detail fidelity for complex tiling textures, and the support ranking system would benefit from learned attention over candidate textures rather than heuristic cosine-similarity ranking.

5.3 Future Work

Several directions appear promising for extending this work:

Alpha-aware training. Incorporating the preprocessed alpha channel into both the model input and the loss function would directly address the glass/transparency failure mode, either through an auxiliary alpha prediction head or through alpha-weighted loss masking.

Learned support selection. Replacing the heuristic ranking with a learned embedding and attention mechanism could improve support quality, particularly for packs with unconventional material families. A contrastive learning approach that maps textures to a shared embedding space would enable cross-pack support retrieval.

Multi-resolution training. Training at multiple resolutions (32×32 and 64×64) or with a super-resolution refinement stage could address the detail fidelity ceiling for complex textures like leaves and bookshelves.

Automatic routing in the ensemble. The current tiered selection logic uses fixed thresholds (0.60, 0.80 structural quality) with a clear margin of 0.10. These thresholds could be learned per-pack or per-texture-family from validation data, or replaced with a lightweight classifier trained on human preference judgments.

Tileability enforcement. Adding a wrap-around consistency loss that penalizes discontinuities at texture boundaries would improve the visual quality of generated textures as repeating tiles in a block-based world.

A Reproduction Instructions

A.1 Environment Setup

The SpriteCraft codebase requires Python 3.13 or later and PyTorch 2.0 or later. Dependencies are managed with uv:

```
git clone https://github.com/RayZh-hs/SpriteCraft
cd SpriteCraft
uv sync
source .venv/bin/activate
```

A.2 Preprocessing

Place the vanilla Minecraft jar file and target resource pack archives (unmodified `.zip` or `.jar`) in `data/raw_packs/`. Optionally create a manifest at `data/manifest.json` to specify pack roles:

Listing A.1: Example manifest.json.

```
{
  "packs": [
    {
      "id": "vanilla",
      "archive": "server-1.21.jar",
      "role": "base",
      "style": "vanilla"
    },
    {
      "id": "Ashen_16x",
      "archive": "Ashen_16x.zip",
      "role": "train",
      "style": "stylized"
    }
  ]
}
```

Run preprocessing:

```
spritecraft preprocess
```

This extracts archives, filters full-cube block textures by resolving blockstate models, resizes images to 32×32 , and writes per-pack datasets to `data/processed/packs/<pack_id>/`. A summary report is written to `data/processed/pack_report.json`.

A.3 Training

Train the full ensemble sequence for all packs:

```
spritecraft train --checkpoint-dir checkpoints/final --steps 10000 --mode recolor_first
```

To train a single stage explicitly, use:

```
spritecraft train --checkpoint-dir checkpoints/final --steps 2500 --mode recolor_only  
spritecraft train --checkpoint-dir checkpoints/final --steps 10000 --mode std_only
```

The `recolor_first` mode trains RecolorNet then StyleAwareUNet sequentially for all packs and passes the recolor checkpoint directory to diffusion validation automatically. Monitor training with TensorBoard:

```
tensorboard --logdir checkpoints/final/tensorboard
```

A.4 Generation

Generate textures for a target pack:

```
spritecraft generate --pack Ashen_16x --checkpoint checkpoints/final --textures stone  
dirt oak_planks
```

Or sample randomly:

```
spritecraft generate --pack Ashen_16x --checkpoint checkpoints/final --random 8
```

Outputs are written to `output/<pack_id>/<bundle>/` containing the generated texture (`produced_tex.png`), vanilla original (`original_tex.png`), style references (`support.png`), comparison image (`comparison.png`), metrics (`metrics.json`), and metadata (`metadata.json`). If the target pack contains the queried texture, a source-of-truth image and MAE/pixel-accuracy metrics are included.

A.5 Repository Layout

Listing A.2: Key project paths.

src/spritecraft/	Python package source
src/spritecraft/models/	Model definitions (unet, recolor, diffusion)
src/spritecraft/training/	Training loops and loss functions
src/spritecraft/inference/	Sampling and generation pipeline
src/spritecraft/data/	Preprocessing, dataset, support ranking
src/spritecraft/debug/	Runtime status and live inspection
data/raw_packs/	Input resource pack archives
data/processed/	Preprocessed per-pack datasets
checkpoints/<run>/	Training checkpoints and validation previews
output/<pack_id>/	Generated texture bundles
report/	LaTeX report source

A.6 Build Notes

The report is configured to build with:

```
make -C report
```

Generated files are written to `report/build/` and ignored by Git.

A.7 Key Configuration Constants

Table A.1: Key configuration constants in `src/spritecraft/config.py`.

Constant	Value
IMAGE_SIZE	32
NUM_TIMESTEPS	40
MIN_SUPPORT_EXEMPLARS	3
MAX_SUPPORT_EXEMPLARS	3
VALIDATION_FILENAMES	20 fixed filenames
MIN_SHARED_PACKS	5

Bibliography

- [1] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.
- [2] Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. Image style transfer using convolutional neural networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2414–2423, 2016.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- [5] Zejiang Hou and Jian Sun. Coordinates matter: High-resolution semantic segmentation for pixel art. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [6] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1125–1134, 2017.
- [7] Justin Johnson, Alexandre Alahi, and Fei-Fei Li. Perceptual losses for real-time style transfer and super-resolution. In *European Conference on Computer Vision*, pages 694–711, 2016.
- [8] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- [9] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171, 2021.
- [10] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.
- [11] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241, 2015.
- [12] Chitwan Saharia, William Chan, Huiwen Chang, Chris Agand Lee, Jonathan Ho, Tim Salimans, David J Fleet, and Mohammad Norouzi. Palette: Image-to-image diffusion models. *ACM SIGGRAPH 2022 Conference Proceedings*, 2022.

- [13] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *International Conference on Learning Representations*, 2021.
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.